ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Master of Science in Data Analytics

MSDA9215 - Big Data Analytics · Final Project, October 2026

Distributed Multi-Model Analytics for E-commerce Data

Technical Report

MongoDB · HBase · Apache Spark

Student name	Godfrey Mushambo
Student ID	101376
Course	Big Data Analytics (MSDA9215)
Lecturer	Temitope Oguntade
Code repository	https://github.com/Mushambo/E-commerce-analytics/tree/main
Submission	10th November 2026

Table of Contents

1. Executive summary **1**

2. System architecture **1**

3. Data modelling and storage (Part 1) **2**

3.1 MongoDB document model 2

3.2 HBase wide-column model 2

3.3 Why each dataset went to MongoDB or HBase 2

4. Spark processing pipeline (Part 2) **2**

5. Integrated analytics (Part 3) **3**

5.1 Customer Lifetime Value - integration/clv_analysis.py 3

5.2 Funnel conversion - integration/funnel_analysis.py 3

5.3 Technology fit and performance 4

6. Visualisations and insights (Part 4) **4**

6.1 Key findings and recommended actions 6

7. Methodology for key analyses **6**

8. Scalability and technology selection **6**

9. Limitations and future work **6**

10. Coverage of the project brief **7**

1. Executive summary

MongoDB stores the business entities (catalogue, customers, orders), HBase stores the high-volume, time-ordered click-stream and daily product metrics, and Spark cleans the data, runs the batch analytics and joins the two stores. All components ran in Docker (MongoDB 7, HBase 2.1.2, Spark 3.5.3). Dataset (10% scale, seed 42): 10,000 users, 5,000 products, 25 categories, 200,000 sessions, 50,000 transactions, 12 Aug - 10 Nov 2026. Scripts and evidence screenshots (S1-S14) are in the separate *Appendices* document; all code is in the GitHub repository linked on the cover.

Net revenue	Sessions with a purchase	Checkout abandonment	CLV held by 17% of customers
$53.2M · 50,000 orders · AOV $1,064	12.6% (25,197 of 200,000)	57% of checkouts (33,018)	27.9% ('Champions')

- **The funnel leaks after the cart.** 98.8% of sessions view a product and 65% add to cart, but only 45% of carts reach checkout and 43% of checkouts become orders - up to ~$25M of unrealised revenue at the $771 average web order.
- **Stock-outs drive the views-sales link.** r = 0.69 across all products but only 0.12 among in-stock products; the 209 sold-out products lost ~60% of their page views.
- **Value is concentrated and partly at risk:** 1,729 Champions hold 27.9% of estimated lifetime value; 1,084 high-value customers have lapsed ~27 days ($4.2M CLV).
- **Revenue is flat, profit is not:** ~$592k/day (s.d. $31k), category revenue within -17%/+15% of the mean, but margins of 18-36% double the profit spread.
- **Reconciling the stores exposes data issues:** 15,552 of 40,749 sessions flagged 'converted' have no order in MongoDB.

2. System architecture

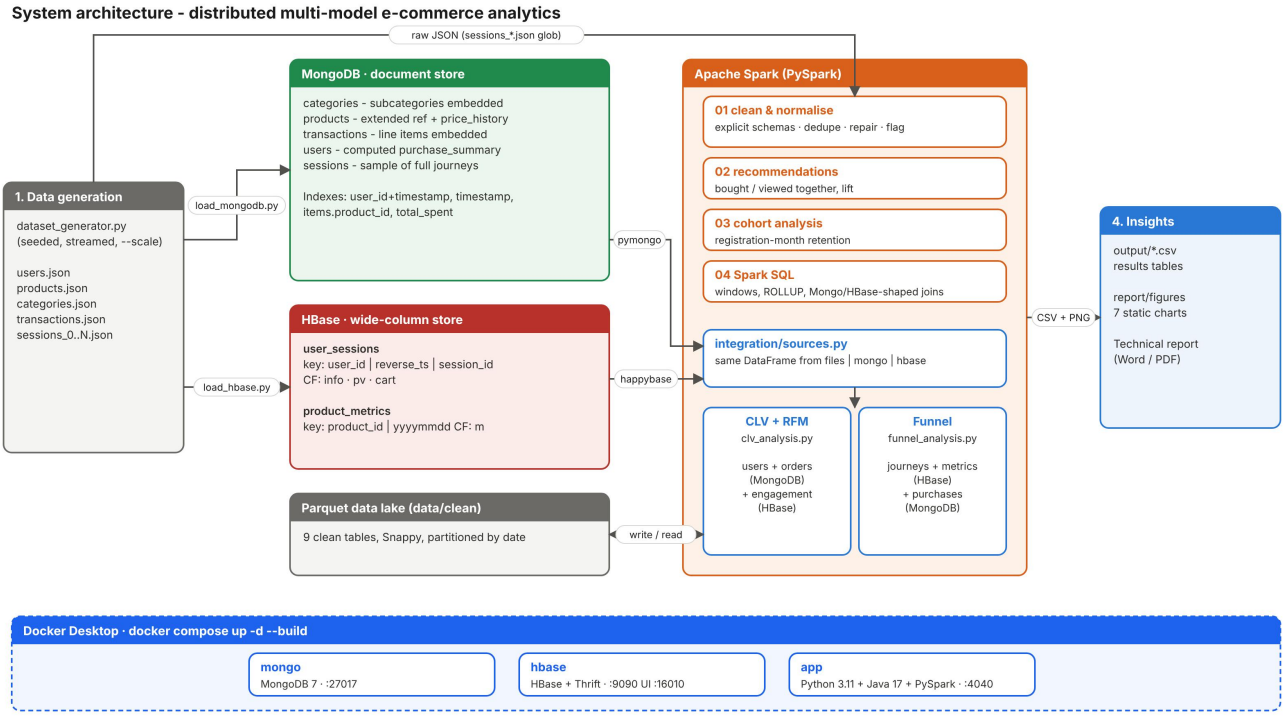


Figure 1 - Architecture: the generator feeds MongoDB, HBase and Spark; Spark joins all sources; everything runs in Docker (containers mongo, hbase, app).

Python loaders write the raw JSON into the two stores; Spark reads the same JSON with explicit schemas and writes a cleaned Parquet lake. Analyses read through one access layer (`integration/sources.py`) that returns identical DataFrames from Parquet, MongoDB (aggregation pushed down to the server) or HBase (row-key range scans), so analysis code is independent of storage. The course generator was kept but fixed: products now get their `subcategory_id`, sessions are streamed to disk in 100k chunks (the original needs >10 GB RAM at full scale), IDs are seeded for reproducibility, and a `--scale` option was added (S4).

3. Data modelling and storage (Part 1)

3.1 MongoDB document model

Collection	Shape	Pattern and justification
categories	category + `subcategories[]` with profit_margin	Embedded hierarchy: small, bounded, always read together.
products	product + `category{id,name}` + `subcategory{id,name,margin}` + `price_history[]`	Extended reference: catalogue pages and margin maths need no $lookup; bounded price history embedded.
transactions	order + `items[]` (product_name, category_id, subcategory_id); BSON Date	Line items embedded: one atomic write per order; revenue by category is a pure $unwind/$group.
users	profile + geo + `purchase_summary{total_spent, order_count, AOV, favourite category, last 5 orders}`	Computed/subset pattern: frequently read facts pre-aggregated; full history stays in transactions.
sessions (50k)	journey with `page_views[]`, `cart_contents[]`	Whole journey in one document for journey queries; full click-stream in HBase.

Indexes match the queries: `{user_id, timestamp}`, `{timestamp}`, multikey `{items.product_id}`, `{purchase_summary.total_spent}`, `{category.category_id, is_active}`. All 120,025 documents loaded in ~10 s (S5). **Four aggregation pipelines** (S6, S7; code in Appendix B.3): (1) **product popularity** - `$unwind` → `$group` → `$sort` → `$lookup` for stock (top product: 84 units, 35 orders); (2) **revenue** and estimated profit by category and month, matching each item to its subcategory margin with `$filter`; (3) **user segmentation** by purchase frequency and spend with `$facet` + `$bucket` (313 users with 10+ orders average $11,449 vs $1,833 for 1-2 orders); (4) **journey funnel** over the embedded `page_views` array by referrer.

3.2 HBase wide-column model

Table	Row key	Column families	Queries served
user_sessions	`user_id|reverse_ts|session_id`, reverse_ts = 9999999999999 - epoch_ms	info session header (15 cells); pv one column per page view; cart one column per product	Latest sessions of a user (prefix scan, newest first); a time window (STARTROW/STOPROW); one session (get)
product_metrics	`product_id|yyyymmdd`	m: views, cart_adds, purchases, units_sold, revenue_cents	A product's history or date range in one contiguous scan; atomic `incr` counters for streaming

Putting `user_id` first keeps each user's activity contiguous; the reversed timestamp returns the newest session first without sorting. Families follow access patterns: listing sessions reads only the small `info` family; the variable-width, sparse `pv` and `cart` families are read only on drill-down. VERSIONS = 1 (immutable events), ROW bloom filters, a 180-day TTL on `pv` and four pre-split points (5 regions per table, S2) complete the design. Loaded: 200,000 session rows and 342,423 product-day rows. Queries for a specific user, e.g. `scan 'user_sessions', {ROWPREFIXFILTER => 'user_000042|', COLUMNS => ['info'], LIMIT => 5}`, are in Appendix B.2 and S8.

3.3 Why each dataset went to MongoDB or HBase

Criterion	MongoDB (catalogue, users, orders)	HBase (sessions, product metrics)
Data shape and volume	Rich nested entities, modest volume	Append-only events, very high volume (~20M page views at full scale), counters
Query pattern	Ad-hoc filters, secondary indexes, aggregation framework	Known key paths: per-user timelines, per-product time ranges
Schema and writes	Flexible documents; atomic per document	Sparse variable columns; very high write rate; automatic region sharding

4. Spark processing pipeline (Part 2)

Cleaning and normalisation (`01_clean_normalize.py`): all JSON files, including the `sessions_*.json` glob, are read with explicit schemas (no second inference pass, no type drift) and written as nine Parquet tables - users, flattened category hierarchy, products with margins, price history, sessions (date-partitioned, with funnel flags), exploded page views and cart items, transactions with reconciliation flags, and line items with pro-rated discounts and margins (broadcast join). Rules: de-duplicate on keys; drop rows with missing keys or bad timestamps; trim and normalise enums and codes; clamp negatives; fill missing descriptive values; repair

conversion_status; and *flag* rather than drop inconsistencies. The data-quality report (S9) found 0 duplicates, 0 invalid rows and 0 orphans, and flagged 15,552 converted sessions without an order.

Recommendation indicators (02_recommendations.py): association rules (support, confidence, lift, top-5 per product via a window) for 'bought together' (self-join on transaction, x < y) and 'viewed together' (pairs built inside each session's array with higher-order functions - no shuffle-heavy join). Only 304 product pairs occur twice or more in 50,000 baskets, so their lifts (median 155) are small-count noise, and category lift is ~0.79 (S10): the synthetic data has no real affinity, which validates the method as a null test. **Cohort analysis** (03_cohort_analysis.py): users grouped by registration month (Feb-Aug) and followed monthly; about 80-84% of every cohort buys in each full month and 90-day revenue per member is flat at $5,253-$5,370 (S11, Figure 7).

Spark SQL (04_spark_sql_analytics.py, S12): CTE + RANK top-3 products per category; ROLLUP funnel by device and referrer; 7-day RANGE window and week-over-week LAG on daily revenue; price-change impact with MAX_BY over exploded price history (≈0.51 → ≈0.49 units/day after any price move - no elasticity); and viewed-vs-sold ranks. To query data that notionally lives in the stores, views shaped like **MongoDB documents** (mongo_users with nested purchase_summary; mongo_transactions with an items array queried via LATERAL VIEW explode, Spark's $unwind) and like **HBase rows** (rowkey + info:* columns, key parsed with SPLIT) are joined in SQL (Appendix B.4).

Optimisations: explicit schemas; Parquet + Snappy with date partitioning (partition pruning); broadcast joins for small dimensions; adaptive query execution; caching only reused DataFrames; array functions instead of joins; pruning rare pairs before ranking; fewer shuffle partitions on one machine. Cleaning took ~20 min on a laptop (S9), dominated by multi-line JSON, which cannot be split across tasks - JSON Lines would remove this bottleneck.

5. Integrated analytics (Part 3)

5.1 Customer Lifetime Value - integration/clv_analysis.py

Business question: which customers are worth most over their expected lifetime, how engaged are they, and who should receive retention spend? **Sources:** user profiles and orders from **MongoDB** ($unwind and discount pro-ration pushed into the pipeline); engagement - session count, duration, pages, cart rate - from **HBase** user_sessions; margins from the category dimension. **Spark:** joins the sources per user, estimates monthly buyer retention from the data (r = 0.830 → expected lifetime 1/(1-r) = 5.9 months), computes CLV = AOV × orders per month × margin × lifetime, and maps RFM quintiles (NTILE windows) to segments (S13).

Segment	Users	Avg CLV	Share of CLV	Recency (days)	Sessions (HBase)	Session→order
Champions	1,729	$4,329	27.9%	3.9	21.1	17.1%
At risk - high value	1,084	$3,910	15.8%	26.7	20.2	14.3%
Loyal	1,260	$3,395	16.0%	9.4	21.1	17.9%
Needs attention	1,939	$2,282	16.5%	8.9	19.6	10.9%
New / promising	1,041	$1,663	6.5%	4.3	19.1	9.1%
Hibernating	2,896	$1,608	17.4%	35.0	19.4	9.5%

Platinum customers (top CLV quartile) average 20.9 sessions and a 16.2% session-to-order rate against 19.3 and 9.0% for Bronze: value comes from **conversion efficiency**, not browsing time (correlation of CLV with session duration 0.02, with session count 0.13).

5.2 Funnel conversion - integration/funnel_analysis.py

Business question: where do shoppers drop out, does it differ by device, traffic source or category, and what is the loss worth? **Sources:** journey stages from **HBase** user_sessions (pv, cart families); category views and cart adds from **HBase** product_metrics; purchases from **MongoDB** transactions as the source of truth. **Spark:** builds per-session stage flags, left-joins orders on session_id, and aggregates by stage, device, referrer, category and week (S14). Result: 200,000 → 197,509 product view → 130,126 cart → 58,215 checkout → 25,197 purchase (Figure 3). Purchase rates differ by < 0.1 point between devices (12.57-12.62%) and < 0.5 between referrers (12.34-12.79%); category view-to-cart rates span only 29.4-30.6%. The loss is a checkout-process problem, not a

channel or assortment problem. Using MongoDB as the source of truth matters: the generator flags 40,749 sessions as converted, but only 25,197 have an order.

5.3 Technology fit and performance

MongoDB serves profiles and orders with server-side aggregation; HBase serves per-user, time-ordered events by key range; Spark performs what neither does well - large cross-store joins and window/statistical computation. A single full-table Thrift scan stalled on single-node HBase, so the reader scans 100 user-id ranges (50 product ranges) and retries a failed range on a new connection (Appendix B.6) - the access path the row key was designed for. At scale, the hbase-spark connector or snapshot export and the MongoDB Spark connector would read partitions in parallel on executors; joins use high-cardinality keys with small dimensions broadcast.

6. Visualisations and insights (Part 4)

Seven static charts (matplotlib, `visualization/make_charts.py`) built from the live-run result tables cover the four kinds of view the brief suggests: sales over time and by category, conversion funnel, product performance, and customer segmentation.

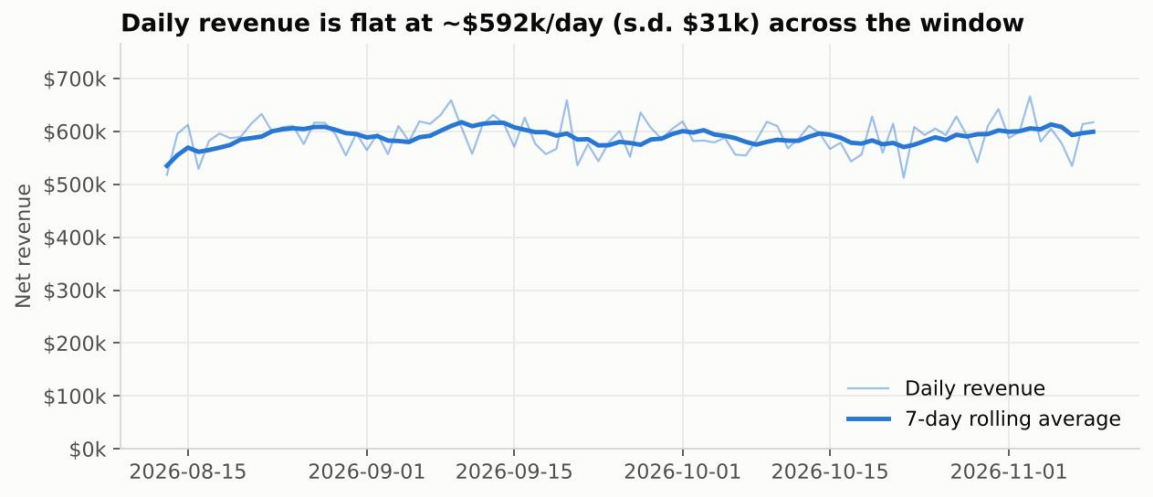


Figure 2 - Sales performance over time: daily net revenue with a 7-day rolling average (Spark SQL window query).

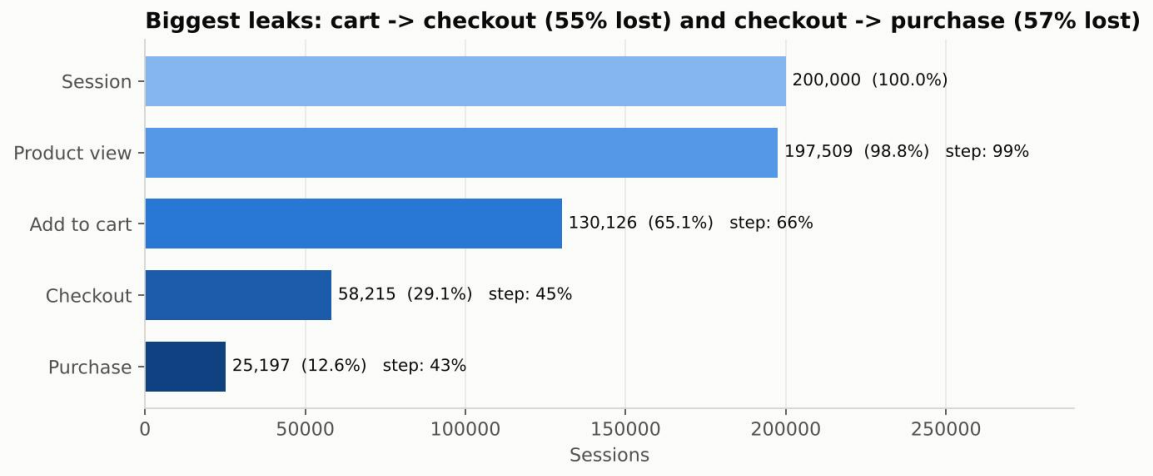


Figure 3 - Conversion funnel stages over 200,000 sessions (HBase journeys × MongoDB orders); step % = conversion from the previous stage.

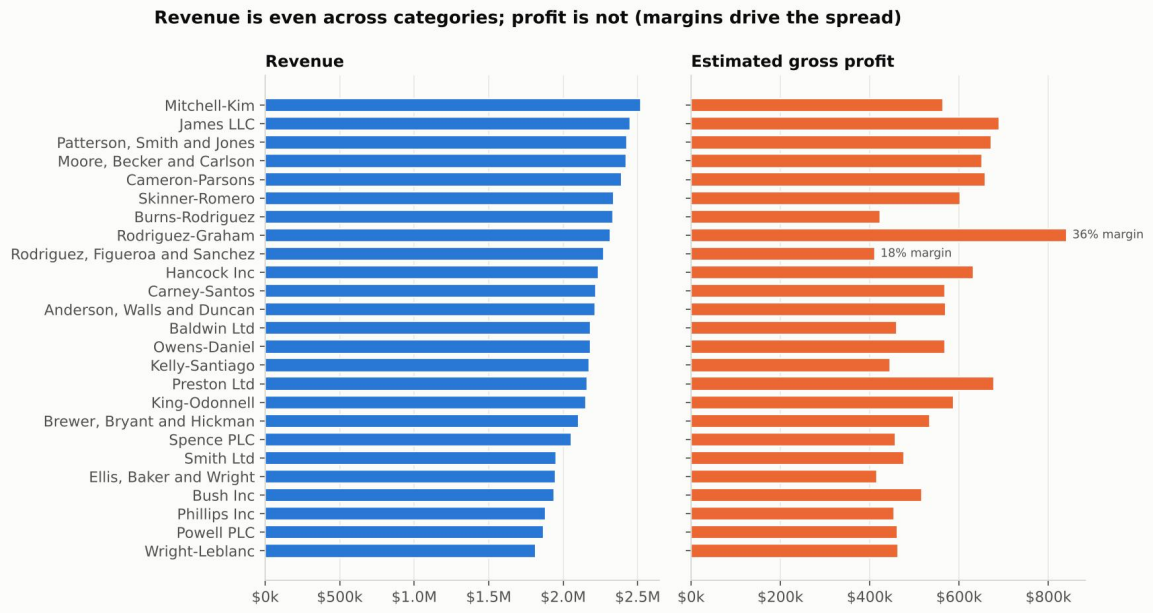


Figure 4 - Sales by category: revenue vs estimated gross profit (MongoDB pipeline 2); margins 18-36%.

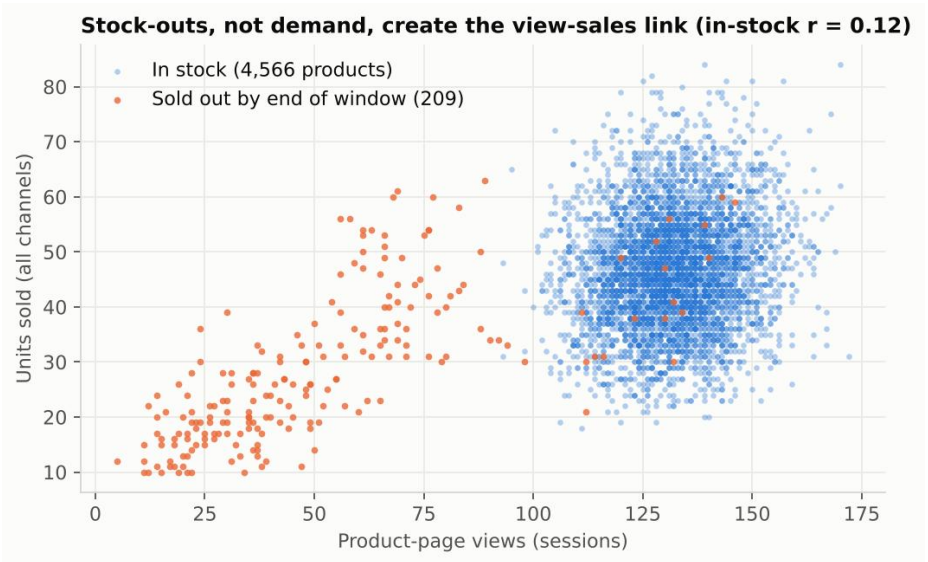


Figure 5 - Product performance, most-viewed vs top-selling: each dot is a product; sold-out products lose page views.

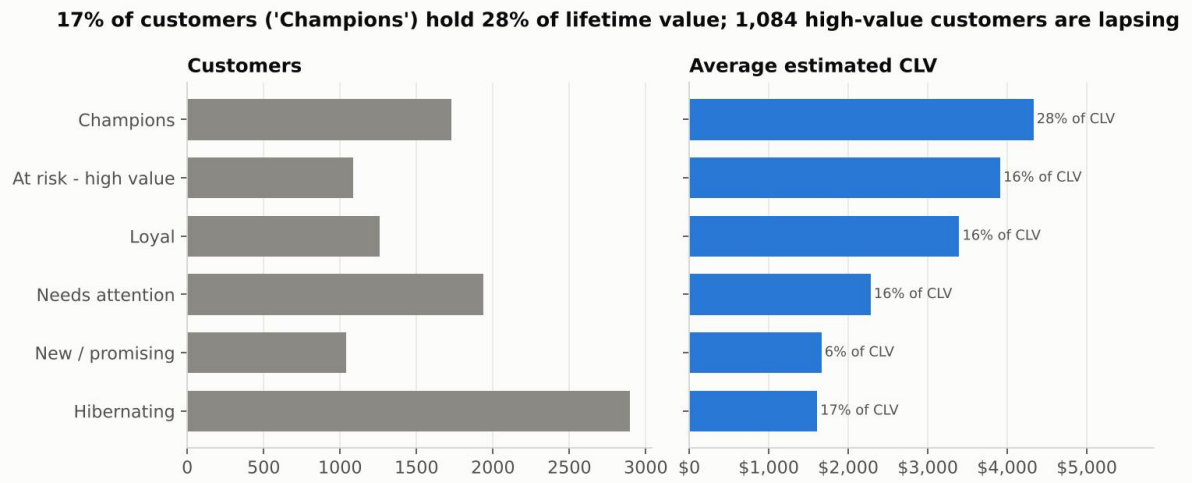


Figure 6 - Customer segmentation: RFM segments by size (left) and average estimated CLV with share of total CLV (right).

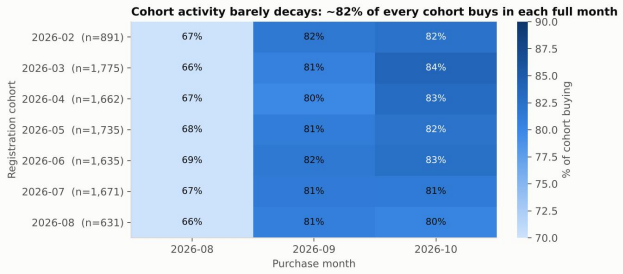


Figure 7 - Cohort analysis: share of each registration cohort buying per month.

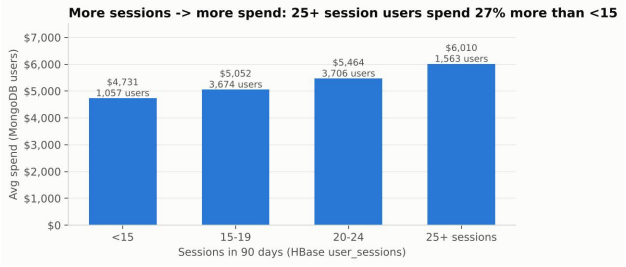


Figure 8 - Engagement (HBase sessions) vs spend (MongoDB users).

6.1 Key findings and recommended actions

- **Fix checkout first** - the largest controllable loss (33,018 sessions). Guest checkout, saved payment methods and cart-recovery e-mails: a 5-point lift adds ~2,900 orders (~$2.2M) per 90 days.
- **Replenish before stock-out** using `product_metrics` views and units - sold-out items lose ~60% of their visibility (Figure 5).
- **Win back 'At risk - high value' customers** (1,084 users, $4.2M estimated CLV) before they hibernate (Figure 6).
- **Merchandise by margin, not revenue** - revenue is even across categories but profit varies two-fold (Figure 4); engaged users (25+ sessions) spend 27% more than those with fewer than 15 (Figure 8).

7. Methodology for key analyses

- **Funnel:** a session reaches a stage if its page sequence contains it; 'checkout' also requires a non-empty cart; 'purchase' requires a MongoDB order with that session_id.
- **CLV:** retention-based model; partial edge months excluded when estimating retention; lifetime capped at 36 months; margin from each item's subcategory. **RFM:** quintiles over buyers, mapped to six named segments.
- **Recommendations:** lift = confidence / P(consequent), minimum pair count 2. **Cohorts:** calendar-month difference between registration and purchase.

8. Scalability and technology selection

Analytical task	Technology and why	Scaling path
Catalogue, profiles, order look-ups and aggregations	MongoDB - nested documents, secondary indexes, aggregation framework	Shard transactions on hashed user_id; replica sets for reads
Click-stream, user timelines, product time series	HBase - key-ordered sparse wide rows, high write rate, counters	Automatic region splits, pre-splitting; salt keys if hot-spots appear
Cleaning, co-occurrence, cohorts, CLV, cross-store joins	Spark - distributed in-memory DataFrames and SQL over all sources	Cluster on YARN/Kubernetes; partitioned Parquet; parallel connectors; Structured Streaming

Processing grows linearly with data: the full dataset (2M sessions, ~4 GB JSON) needs only more partitions. Pair generation is the only super-linear step and is bounded by the 15-page session cap. In production, events would arrive via Kafka, update HBase counters through Spark Structured Streaming, and be compacted nightly to Parquet.

9. Limitations and future work

- **Synthetic data has little behavioural signal:** uniform devices, referrers and categories; no cohort decay; no price elasticity; product-pair rules suffer from small counts. Real data would need support thresholds, significance tests or collaborative filtering (Spark ALS).
- **Generator artefacts:** once the transaction target is reached, converted sessions stop producing orders (the 15,552 gap); half the orders have no session; there is no age field, so segmentation is behavioural rather than by age group.
- **Infrastructure:** single-node HBase in Docker was the weak point - it stopped once under memory pressure and stalled on long scans, which led to the retrying range-scan reader. **Future work:** JSON Lines input, Spark ALS, streaming counters, probabilistic (BG/NBD) CLV, an interactive dashboard.

10. Coverage of the project brief

Brief requirement	Where it is answered
P1 MongoDB: catalogue with category hierarchy, user profiles with embedded summary, transactions with line items; loaded subset; ≥ 2 non-trivial pipelines (popularity, segmentation, revenue)	§3.1, Appendix B.3, S5-S7 (4 pipelines)
P1 HBase: column families for time-series sessions and product metrics; loaded subset; queries for a specific user; schema commands	§3.2, Appendix B.1-B.2, S2, S8
P1 design justification and MongoDB vs HBase comparison	§3.1-3.3
P2 Spark cleaning/normalisation; recommendation indicators or cohort analysis (both done)	§4, S9-S11
P2 Spark SQL complex queries + querying MongoDB/HBase-shaped data; pipeline explanation and optimisations	§4, Appendix B.4, S12
P3 integrated query: business question, sources, processing steps, Spark's role, technology fit, performance (two queries: CLV and funnel)	§5.1-5.3, Appendix B.5-B.6, S13-S14
P4 ≥ 3-4 static visualisations: sales over time and by category, segmentation, top-selling vs most-viewed, funnel	§6, Figures 2-8
Report checklist: architecture · modelling · Spark · integration · scalability · technology selection · findings · visualisations · methodology · limitations	§2 · §3 · §4 · §5 · §8 · §8 · §1, 6.1 · §6 · §7 · §9
Repository: all code, sample queries, reproduction instructions	GitHub (link on the cover), README.md, Appendix A